SIMATS ENGINEERING

CO3-AT3-ASSIGNMENT-2

COURSE CODE: CSA0904 **NAME:** Rose Mystica. J

COURSE NAME: Programming in JAVA **REG NO:** 192421410

SIMATS ENGINEERING

SIMATS Online Programming Lab

JAVA PROGRAMMING

ROSE MYSTICA J
192421410RA

CO3-AT3-Assignment-2

← Back

QUESTIONS

Q1
Online Ticket Booking ◆ Thread Synchronization
25 marks

Q2
ATM Transaction System ◆ Multiple Threads and Exceptions
25 marks

Q3
Producer-Consumer ◆ Inter-Thread Communication
25 marks

Q4
Product Price Input ◆ NumberFormatException
25 marks

4/4 answered

Q1 Online Ticket Booking ◆ Thread Synchronization 25 marks

Problem Statement:
Develop a multithreaded Java program to simulate an online ticket booking system. Multiple users should attempt to book seats simultaneously.

Requirements:

Create multiple booking threads.
Maintain a common number of available seats.
Use synchronization to prevent multiple threads from booking the same seat.
Display successful and unsuccessful booking attempts.

Your Code

```
class Main {
    public static void main(String[] args) throws Exception {
        final int[] seats = {3};
        Runnable booking = () -> {
            synchronized (seats) {
                String name = Thread.currentThread().getName();
                if (seats[0] > 0) {
                    seats[0]--;
                    System.out.println(name + " booked a seat");
                } else {
                    System.out.println(name + " booking failed");
                }
            }
        };
        Thread t1 = new Thread(booking, "User 1");
        Thread t2 = new Thread(booking, "User 2");
        Thread t3 = new Thread(booking, "User 3");
        Thread t4 = new Thread(booking, "User 4");
        t1.start();
        t2.start();
        t3.start();
        t4.start();
        t1.join();
        t2.join();
        t3.join();
        t4.join();
        System.out.println("Available seats: " + seats[0]);
    }
}
```

Input

```
stdin input
```

Output

```
User 1 booked a seat
User 4 booked a seat
User 3 booked a seat
User 2 booking failed
Available seats: 0
```

▶ Run Save

SIMATS ENGINEERING

SIMATS Online Programming Lab

JAVA PROGRAMMING

ROSE MYSTICA J
192421410RA

CO3-AT3-Assignment-2

← Back

QUESTIONS

- Q1 ✓ Online Ticket Booking ◆ Thread Synchronization — 25 marks
- Q2 ✓ ATM Transaction System ◆ Multiple Threads and Exceptions — 25 marks
- Q3 ✓ Producer-Consumer ◆ Inter-Thread Communication — 25 marks
- Q4 ✓ Product Price Input ◆ NumberFormatException — 25 marks

4/4 answered

Q2 ATM Transaction System ◆ Multiple Threads and Exceptions — 25 marks

Problem Statement:
Develop a Java program to simulate an ATM system where multiple transactions are performed concurrently on a bank account.

Requirements:

Create separate threads for withdrawal and deposit.
Synchronize access to the account.
Create an InsufficientBalanceException.
Handle invalid withdrawal attempts.
Display the final account balance.

Your Code

```
class Main {
    static class InsufficientBalanceException extends Exception {
        InsufficientBalanceException(String msg) {
            super(msg);
        }
    }
    static int balance = 1000;
    static synchronized void deposit(int amount) {
        balance += amount;
        System.out.println("Deposited: " + amount);
    }
    static synchronized void withdraw(int amount) {
        try {
            if (amount > balance)
                throw new InsufficientBalanceException(
                    "Insufficient balance for withdrawal: " + amount);
            balance -= amount;
            System.out.println("Withdrawn: " + amount);
        } catch (InsufficientBalanceException e) {
            System.out.println(e.getMessage());
        }
    }
    public static void main(String[] args) throws Exception {
        Thread t1 = new Thread(() -> deposit(500));
        Thread t2 = new Thread(() -> withdraw(700));
        Thread t3 = new Thread(() -> withdraw(1000));
        Thread t4 = new Thread(() -> withdraw(500));
        t1.start();
        t1.join();
        t2.start();
        t2.join();
        t3.start();
        t3.join();
        t4.start();
        t4.join();
        System.out.println("Final Balance: " + balance);
    }
}
```

Input

```
stdin input
```

Output

```
Deposited: 500
Withdrawn: 700
Insufficient balance for withdrawal: 1000
Withdrawn: 500
Final Balance: 300
```

▶ Run Save

SIMATS ENGINEERING

SIMATS Online Programming Lab

JAVA PROGRAMMING

ROSE MYSTICA J
192421410RA

CO3-AT3-Assignment-2

← Back

QUESTIONS

- **Q1** Online Ticket Booking ◆ Thread Synchronization — 25 marks
- **Q2** ATM Transaction System ◆ Multiple Threads and Exceptions — 25 marks
- **Q3** Producer-Consumer ◆ Inter-Thread Communication — 25 marks
- **Q4** Product Price Input ◆ NumberFormatException — 25 marks

4/4 answered

Q3 Producer-Consumer ◆ Inter-Thread Communication — 25 marks

Problem Statement:
Develop a Java program to implement a Producer-Consumer scenario using inter-thread communication.

Requirements:

Create a producer thread and a consumer thread.
Use a shared buffer.
Use wait() and notify() for communication.
Ensure that the consumer waits when the buffer is empty.
Ensure that the producer waits when the buffer is full.

Your Code

```
class Main {
    static int item = 0;
    static boolean available = false;
    static synchronized void produce(int n) throws Exception {
        while (available)
            Main.class.wait();
        item = n;
        System.out.println("Produced: " + n);
        available = true;
        Main.class.notify();
    }
    static synchronized void consume() throws Exception {
        while (!available)
            Main.class.wait();
        System.out.println("Consumed: " + item);
        available = false;
        Main.class.notify();
    }
    public static void main(String[] args) {
        Thread producer = new Thread(() -> {
            try {
                for (int i = 1; i <= 5; i++)
                    produce(i);
            } catch (Exception e) {
                System.out.println(e);
            }
        });
        Thread consumer = new Thread(() -> {
            try {
                for (int i = 1; i <= 5; i++)
                    consume();
            } catch (Exception e) {
                System.out.println(e);
            }
        });
        producer.start();
        consumer.start();
    }
}
```

Input

```
stdin input
```

Output

```
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
```

▶ Run Save

SIMATS ENGINEERING

SIMATS Online Programming Lab

JAVA PROGRAMMING

ROSE MYSTICA J
192421410RA

CO3-AT3-Assignment-2

← Back

QUESTIONS

- Q1 Online Ticket Booking ◆ Thread Synchronization — 25 marks
- Q2 ATM Transaction System ◆ Multiple Threads and Exceptions — 25 marks
- Q3 Producer-Consumer ◆ Inter-Thread Communication — 25 marks
- Q4 Product Price Input ◆ NumberFormatException — 25 marks

4/4 answered

Q4 Product Price Input ◆ NumberFormatException

25 marks

Problem Statement:
Develop a Java program that accepts the price of a product as a String and converts it into a numeric value. Handle invalid numeric input without terminating the program.

Requirements:

Read product price as a String.
Convert it using Integer.parseInt() or Double.parseDouble().
Handle NumberFormatException.
Display an appropriate error message.

Your Code

```java
import java.util.Scanner;
class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String price1 = sc.nextLine();
        String price2 = sc.nextLine();
        try {
            double p1 = Double.parseDouble(price1);
            System.out.println("Product Price: " + p1);
        } catch (NumberFormatException e) {
            System.out.println("Invalid price: " + price1);
        }
        try {
            double p2 = Double.parseDouble(price2);
            System.out.println("Product Price: " + p2);
        } catch (NumberFormatException e) {
            System.out.println("Invalid price: " + price2);
        }
        System.out.println("Program completed successfully");
    }
}
```

Input

```
499.50
abc
```

Output

```
Product Price: 499.5
Invalid price: abc
Program completed successfully
```

▶ Run Save